Angular 22 Portfolio

Step-by-Step Build Guide

Scaffolding • Component Deep-Dives • Architecture Diagrams

Prepared for Paul Randal
Founding Software Engineer • Remote Health Monitoring Platform
October 2026

This guide walks through building a professional portfolio site in Angular 22 to serve as recent technical signal for a software career restart. It includes the recommended scope, full scaffolding, in-depth component design, and visual architecture diagrams.

1. Overview & Goals

The portfolio is Task 1 of the career restart plan. It creates a public, living artifact that demonstrates ownership, architecture judgment, and recent activity. It is not meant to be feature-rich on day one — it is meant to be high-quality, deployable quickly, and easy to iterate on while you refresh Android/Kotlin and ship new projects.

Why this works as the first move

- Hands-on practice with modern Angular (standalone components, zoneless, OnPush defaults) without a long ramp-up.
- Creates the public URL you will link from resumes and LinkedIn.
- Forces clear case-study writing — the same skill needed in interviews.
- Can be finished to a live skeleton in days, then polished over 1–2 weeks in parallel with refresher work.

Recommended scope

Keep the first version simple and high-quality. A multi-page Angular 22 application with clear sections is enough. Host on Netlify, Vercel, Firebase Hosting, or GitHub Pages.

2. Step-by-Step Build Plan

Week 0 — Decide & Scaffold

Confirm Angular 22. Create the project (or use the provided starter). Initialize Git and push to a professional GitHub account. Decide hosting early so the deploy path is clear.

Day 1–2 — Core Layout & Routing

Build the site shell (header, mobile nav, dark mode toggle, footer). Wire all routes. Deploy a navigable skeleton even if content is placeholder.

Day 2–4 — Content & Case Studies

Write the About section (lead with founding role). Populate the Project model with real or cleaned past projects. Create at least one polished project detail page.

Day 4–6 — Polish & Signal

Add screenshots or short demos. Add Open Graph meta tags. Add a clear “open to opportunities” line. Tighten skills and contact.

Ongoing

Iterate design. Add new projects as you ship them. Keep the site live and linked from LinkedIn/resume.

3. Scaffolding (Angular 22)

The starter uses Angular 22 standalone components, zoneless change detection (default since v21), and modern control flow (@for, @if). OnPush is the default for newly generated components.

Folder structure

```
portfolio/
+-- angular.json
+-- package.json                # Angular 22.2.x
+-- tsconfig.json
+-- tsconfig.app.json
+-- README.md
+-- src/
    +-- index.html
    +-- main.ts
    +-- styles.scss             # global + dark mode
    +-- favicon.ico
    +-- assets/                 # screenshots, resume PDF
    +-- app/
        +-- app.component.ts    # shell + nav + theme
        +-- app.routes.ts
        +-- models/
        |   +-- project.model.ts
        +-- features/
            +-- home/
            +-- about/
            +-- skills/
            +-- projects/
            |   +-- projects.component.ts
            |   +-- project-detail.component.ts
            +-- resume/
            +-- blog/
            +-- contact/
```

Figure 1. Complete Angular 22 portfolio folder structure.

Key package versions

Package	Version
@angular/core (and related)	^22.2.3
@angular/cli	^22.2.3
typescript	~5.8.x
rxjs	~7.8.0

Note: Zone.js is not required (zoneless default).

Quick start

```
cd portfolio
npm install
npm start
# open http://localhost:4200
```

4. Architecture Diagrams

High-level application architecture

The application is a classic standalone Angular SPA. A single root component owns the shell (header, navigation, footer). Child routes render feature components into the router-outlet. All project data lives in a single typed model that both the list and detail views consume.

Layer	Responsibility	Key Files
Shell	Header, mobile nav, dark mode, footer, router outlet	app.component.ts
Routing	URL → component mapping, titles, wildcards	app.routes.ts
Features	Page-level UI and behavior	features/*/*.component.ts
Data	Typed project model + static data	models/project.model.ts
Assets & Styles	Images, PDF, global CSS, dark mode	assets/, styles.scss

Figure 2. Application layers and responsibilities.

Routing map

Path	Component	Purpose
/	HomeComponent	Hero + featured projects
/about	AboutComponent	Bio + founding story
/skills	SkillsComponent	Grouped skills
/projects	ProjectsComponent	Project grid
/projects/:slug	ProjectDetailComponent	Full case study
/resume	ResumeComponent	PDF download
/blog	BlogComponent	Optional writing
/contact	ContactComponent	Email, GitHub, LinkedIn

Figure 3. Complete routing table.

5. Component Deep Dives

AppComponent (Shell)

Owns the persistent layout: logo, primary navigation, mobile hamburger menu, dark-mode toggle, and footer. Uses signals for menu and theme state. Applies the dark class to the document root.

Notes: Key behaviors: responsive nav (hidden links + hamburger below 768px), active link styling via routerLinkActive, theme toggle that toggles a CSS class.

HomeComponent

Hero section with name, one-line positioning, and primary CTAs (View Work / Get in Touch). Below the hero, a grid of featured projects pulled from the Project model (filter on featured: true).

Notes: Keep the hero concise. The featured section should surface your strongest case study (remote health monitoring).

AboutComponent

Short, factual bio that leads with the BYU CS degree and the founding software role. Covers the full product (Angular portals, Android/iOS Bluetooth apps, HIPAA Firebase), team leadership, 2024 exit, and current focus.

Notes: Write in first person. Aim for 80–140 words total. Avoid generic “passionate developer” language.

SkillsComponent

Skills grouped by how you actually work: Languages & Frameworks, Mobile & Backend, Practice & Tools. No progress bars or logo walls.

Notes: This page exists so hiring managers can quickly scan relevant experience without digging through project write-ups.

ProjectsComponent

Responsive grid of project cards. Each card shows title, short description, tech tags, and a link to the detail page. Data comes from the shared PROJECTS array.

Notes: Cards should be scannable. Use real screenshots when available.

ProjectDetailComponent

Full case study page. Sections: Problem, Role & Decisions, Tech stack (tags), Outcomes, and optional Live / Code links. Resolves the project by slug from the route.

Notes: This is where architecture judgment is demonstrated. Be specific about tradeoffs.

ResumeComponent

Simple page with a prominent download button pointing to a PDF in assets/.

Notes: Keep the resume itself focused on the startup ownership story and recent signal.

BlogComponent

Optional placeholder. Use only if you plan to write. Otherwise leave as a light “coming soon” or remove the route.

Notes: Do not force content. Quality of project case studies matters more.

ContactComponent

Email, GitHub, and LinkedIn links. Short note that you are open to senior, Android-focused, full-stack, or solutions-engineering conversations.

Notes: Update the placeholder links before deploying.

6. Project Data Model

A single typed interface drives both the project list and the detail pages. This keeps content and presentation separated and makes it easy to add or update projects.

```
export interface Project {
  id: string;
  slug: string;
  title: string;
  shortDescription: string;
  problem: string;
  roleAndDecisions: string;
  techStack: string[];
  outcomes: string;
  image?: string;
  liveUrl?: string;
  repoUrl?: string;
  featured?: boolean;
}
```

Figure 4. Project interface (src/app/models/project.model.ts).

The sample data already contains a full entry for the Remote Health Monitoring Platform (Angular portals, Android/iOS Bluetooth, HIPAA Firebase, team of 8). Replace the placeholders with your other past projects and, later, the new Android/AI projects you build.

7. High-Leverage Additions

The scaffolding covers the core pages. These additions most improve the site as a hiring signal:

Real screenshots or short demos — A clean image of the portals or Android app is more convincing than text alone.

Open Graph / meta tags — Professional LinkedIn preview when the URL is shared.

“Open to opportunities” line — Reduces friction in the hero or about section.

Quantified outcomes — Even approximate numbers (team size, data volume, growth to 30 employees) strengthen case studies.

Proper favicon + social image — Small polish that makes the site feel finished.

Simple contact form — Optional; increases replies for people who prefer not to email cold.

Skip link + accessibility basics — Focus states, contrast, semantic landmarks.

8. Deployment

Deploy an early version as soon as the skeleton is navigable. A live URL is more useful than a local-only project. Common options that work well with Angular 22:

- Netlify — drag-and-drop or Git integration, excellent for static builds.
- Vercel — similarly straightforward with Angular.
- Firebase Hosting — especially convenient given your prior Firebase experience.
- GitHub Pages or S3 — simple and free.

After the first deploy, continue polishing content and design. The portfolio becomes the living home for the past projects you upload and the new Android/AI projects you ship over the next 8–12 weeks.

Next Actions

1. Clone or copy the starter and run npm install && npm start.
2. Replace placeholder project data with your real past projects.
3. Update contact links and add a resume PDF to assets/.
4. Add at least one real screenshot.
5. Deploy a live version this week.
6. Continue Android refresher and new project work in parallel.

You already have the Angular experience and the product-ownership story. This portfolio turns that into a clear, public, recent signal.